

Безопасность приложений: лабораторные работы

Оглавление

Правила проведения лабораторных работ.....	4
0. Ключевые правила	4
1. Что нужно предоставить в качестве решения.....	4
Шаблон доказательства (копировать/вставлять для каждой лабораторной работы).....	4
2. Среда для выполнения лабораторных работ (OWASP Juice Shop*).....	4
3. Индекс лабораторных работ (соответствие OWASP Top 10).....	5
4. Формат лабораторного занятия (как работать со студентами на занятиях)	5
ЛР_0 — Инвентаризация активов и критические активы (поиск в Juice-Shop).....	6
Цели	6
Шаги.....	6
Таблица результата (заполняется обучающимися).....	6
ЛР_1 — Пакет контроля доступа	8
Часть 1 — Поиск административного маршрута (концепция вертикального контроля доступа)....	8
Часть 2 — Просмотр корзины другого пользователя (горизонтальный контроль доступа / IDOR) .	8
Вопросы для обсуждения.....	8
Список проверок для предотвращения.....	8
ЛР_2 — Устаревший интерфейс (оставленная поверхность атаки)	10
Шаги.....	10
Дополнительно (по желанию) — Утечка информации через обработку ошибок.....	10
Список проверок для предотвращения.....	10
ЛР_3 — Уязвимая библиотека (резервный файл + сообщение об уязвимости).....	11
Часть 1 — Получение забытого резервного файла.....	11
Часть 2 — Выявление уязвимой зависимости и её сообщение	11
Список проверок для предотвращения.....	11
ЛР_4 — Утечка конфиденциальных документов через публичный файловый раздел + неподписанный JWT.....	12
Часть 1 — Утечка конфиденциальных документов через публичный файловый раздел	12
Шаги.....	12
Вопросы для обсуждения.....	12
Список проверок для предотвращения.....	12
Часть 2 — Неподписанный JWT-токен.....	12

Шаги	12
Список проверок для предотвращения	13
ЛР_5 — Пакет инъекций (SQL-инъекция + DOM-XSS)	14
Часть 1 — SQL-инъекция при входе (обход аутентификации)	14
Часть 2 — DOM-XSS через поиск	14
Список проверок для предотвращения	14
ЛР_6 — Время расплаты (отрицательное количество → злоупотребление бизнес-логикой)	15
Шаги	15
Вопросы для обсуждения	15
Список проверок для предотвращения	15
ЛР_7 — Слабые/дефолтные учетные данные (без SQL-инъекции)	16
Шаги	16
Список проверок для предотвращения	16
ЛР_8 — небезопасная десериализация (бомба памяти)	17
Шаги	17
Почему это работает	17
Список проверок для предотвращения	18
ЛР_9 — Открытые метрики (утечка эндпоинта (конечной точки) наблюдаемости)	19
Шаги	19
Список проверок для предотвращения	19
ЛР_10 (опционально) — Микролаборатория SSRF (два контейнера — docker-compose)	20
Запуск среды для выполнения лабораторной работы	20
Шаги	22
Список проверок для предотвращения	22
ЛР_11 — Инъекция команд через переполнение буфера (коррупция данных → выполнение команд)	23
Цели	23
Модель угроз	23
Часть 1 — Сборка и базовый запуск	23
Часть 2 — Триггер выполнения команды через переполнение (только для обучения)	24
Часть 3 — Проверка исправления	24
Вопросы для обсуждения	24
Список проверок для предотвращения	24
Исходный код	25
`overflow_cmd.c` — намеренно уязвимый (переполнение затрагивает соседний `cmd`)	25
`overflow_cmd_fixed.c` — исправленная версия (корректные границы, отсутствие влияния атакующего)	26
Цели `Makefile` (добавьте в ваш Makefile)	27

Правила проведения лабораторных работ

0. Ключевые правила

- Тестировать методики только в учебной среде, предоставленной для лабораторных работ.
- Не использовать содержимое запросов (payload) и изученные методы против реальных систем.
- Если что-то сломалось, обновите страницу, перезапустите контейнер или откройте новую инкогнито-сессию.

1. Что нужно предоставить в качестве решения

Для каждой лабораторной работы фиксируйте:

- Скриншот с решённой задачей (таблицу результатов) или иной явный индикатор успеха.
- Ключевой артефакт: ссылку URL, состояние страницы, фрагмент запроса/ответа, конфигурацию или содержимое файла.
- Заполните «Шаблон решения».

Шаблон решения (копировать/вставлять для каждой лабораторной работы)

- Номер лабораторной работы и её название: _____
- Что я сделал (кратко): _____
- Доказательства (скриншоты/заметки): _____
- Ключевая причина возникновения угрозы: _____
- Потенциальные последствия: _____
- Исправление / профилактика: _____

2. Среда для выполнения лабораторных работ (OWASP Juice Shop*)

1. Запустите приложение Juice Shop локально (стандартный Docker-образ):

```
```bash
```

```
docker run --rm -p 3000:3000 -e NODE_ENV=unsafe bkimminich/juice-shop
```

```
```
```

2. Откройте в браузере: <http://localhost:3000>

Рекомендация: каждый студент запускает собственный экземпляр – так проще сбрасывать состояние и избегать взаимных помех.

*Это программное обеспечение распространяется бесплатно: вы можете распространять и/или модифицировать его в соответствии с условиями лицензии MIT. OWASP Juice Shop и все вклады защищены авторским правом Bjoern Kimminich и участников проекта 2014-2021.

3. Индекс лабораторных работ (соответствие OWASP Top 10)

Мы следуем порядку OWASP Top 10 (2025) от A01 до A10.

Подготовительная (нулевая) лабораторная работа формирует обзор активов приложения, после чего переходим к каждому пункту поочерёдно.

4. Формат лабораторного занятия (как работать со студентами на занятиях)

Каждая лаборатория проходит в одинаковом ритме, чтобы курс оставался ориентированным на концепции, а не на инструменты.

- **Контекст** (5–7 мин): определение категории + реальный сценарий – объясняется во время презентации.
- **Выполнение лабораторной работы** (20–25 мин): следуем списку шагов и фиксируем доказательства.
- **Разбор** (10–12 мин): коренная причина → паттерн профилактики → план верификации.

ЛР_0 — Инвентаризация активов и критические активы (поиск в Juice-Shop)

Подготовительная лабораторная работа охватывает активы, точки входа и границы доверия.

Цели

- Выявить активы (данные + функциональность) и ранжировать их по критичности.
- Определить точки входа и границы доверия.
- Сформулировать топ-5 критических активов, которые необходимо защищать в течение жизненного цикла разработки (SDLC).

Шаги

1. Пройдитесь по приложению и просмотрите каталог товаров → зарегистрируйтесь → войдите в систему → добавьте товары в корзину → откройте страницы оформления/оплаты → зайдите в раздел «About/Contact».
2. Составьте инвентарь активов, включив:
 - a. Идентификации/роли,
 - b. Сессии/токены,
 - c. Заказы/корзину,
 - d. Процесс оплаты,
 - e. Административные функции,
 - f. Файлы/загрузки,
 - g. API,
 - h. Телеметрию/метрики/логи.
3. Сопоставьте точки входа: UI-маршруты, REST-эндпоинты (`/api`, `/rest`), файловые эндпоинты (например, `/ftp`), всё, что можно обнаружить через клиентский JavaScript.
4. Определите границы доверия: Браузер ↔ Веб-приложение; Веб-приложение ↔ База данных; Веб-приложение ↔ Сторонние сервисы; потоки только для администраторов ↔ обычных пользователей.
5. Выберите «Топ-5 критических активов» и опишите, каковы последствия их компрометации

Таблица результата (заполняется обучающимися)

| Актив | Влияние CIA
(высокое /
среднее /
низкое) | Кто должен
иметь доступ | Точки входа | Приоритет
защиты |
|------------------------------------|---|---|--|---------------------|
| (Пример)
Персональные
данные | Высокое | Пользователи —
только к своим
данным

Персонал
магазина —
доступ по
принципу
«минимально
необходимого» | /profile
/privacy-
security/data-
export
... | 1 |
| | | | | |
| | | | | |

ЛР_1 — Пакет контроля доступа

Лабораторная работа охватывает первый риск OWASP Top 10:2025 A01 — Нарушение контроля доступа.

Часть 1 — Поиск административного маршрута (концепция вертикального контроля доступа)

Шаги

1. Откройте «DevTools → Sources».
2. Поищите какие из файлов приложения могут быть полезны для определения скрытых ресурсов
3. Используя файлы или иным способом найдите административные ресурсы приложения, инвентаризируйте их и попробуйте получить доступ
4. Зафиксируйте поведение и сделайте скриншот.

Часть 2 — Просмотр корзины другого пользователя (горизонтальный контроль доступа / IDOR)

Шаги

1. Войдите в систему как обычный пользователь и добавьте 1-2 товара в корзину.
2. Откройте «DevTools → Application (Storage)».
3. Найдите переменную, отвечающую за идентификатор корзины.
4. Проверьте наличие валидации доступа к существующим «корзинам» и попробуйте получить доступ к корзине другого пользователя.
5. При необходимости выполните «жёсткое» обновление страницы.
6. Зафиксируйте доказательство доступа к чужой корзине (или скриншот решённого задания в таблице результатов).

Вопросы для обсуждения

- Где обязательно должна происходить авторизация (пользовательский интерфейс (UI), интерфейс программирования приложений (API), база данных)?
- Какие идентификаторы сервер никогда не должен принимать от клиента как достоверные?

Список проверок для предотвращения

- Выполнять проверку авторизации на сервере для каждого объекта (проверка владения).

- Избегать прямых ссылок на объекты без авторизации.
- Предпочитать случайные (opaque) идентификаторы, чтобы снизить предсказуемость.

ЛР_2 — Устаревший интерфейс (оставленная поверхность атаки)

Лабораторная работа охватывает второй риск OWASP Top 10:2025 A02 — Нарушение безопасности конфигураций.

Шаги

1. Войдите в систему под любой учётной записью.
2. Перейдите в «Contact Us → Complain?» (форма подачи жалобы).
3. Обратите внимание, что в списке разрешённых типов файлов указаны только определенные, например, PDF и ZIP.
4. Используя инструменты DevTools попробуйте определить какие еще расширения файлов может принять API с помощью устаревшего интерфейса
5. Попробуйте отправить жалобу, приложив файл недопустимого расширения
6. Зафиксируйте доказательство её принятия и/или отметку о выполнении задания.

Дополнительно (по желанию) — Утечка информации через обработку ошибок

Откройте несуществующий REST-эндпоинт и посмотрите, возвращает ли сервер подробные сообщения об ошибке:

Список проверок для предотвращения

- Удалять неиспользуемые/устаревшие эндпоинты; применять списки разрешённых путей на границе (gateway/WAF — последний уровень защиты).
- Выполнять серверную проверку загружаемых файлов по содержимому (magic bytes), а не только по расширению или MIME-типу.

Обеспечить единообразную обработку ошибок: не выводить стек-трейсы и внутренние детали в ответах пользователям.

ЛР_3 — Уязвимая библиотека (резервный файл + сообщение об уязвимости)

Лабораторная работа охватывает третий риск OWASP Top 10:2025 A03 — Уязвимости в цепочке поставок ПО.

Часть 1 — Получение забытого резервного файла

Откройте FTP-каталог: <http://localhost:3000/ftp>

Попробуйте открыть резервную копию ``package.json`` (часто блокируется по типу файла).

Подсказка: попробуйте использовать обход с «Poison Null Byte»:

Часть 2 — Выявление уязвимой зависимости и её сообщение

1. В скачанном резервном ``package.json`` просмотрите список зависимостей и их версии.
2. Найдите хотя бы одну зависимость с известной уязвимостью (например, ``sqlite``).
3. Зафиксируйте доказательство (отметка в табло результатов или подтверждающий ответ).

Список проверок для предотвращения

- Генерировать SBOM и применять политики зависимостей (фиксация версий, allow-/deny-листы).
- Не хранить чувствительные файлы вроде ``package.json`` рядом с рабочим каталогом приложения.
- Автоматизировать сканирование уязвимостей в CI и отслеживать сроки исправления (SLA).
- Предотвращать попадание артефактов разработки/резервных копий в production-образы.

ЛР_4 — Утечка конфиденциальных документов через публичный файловый раздел + неподписанный JWT

Лабораторная работа охватывает четвёртый риск OWASP Top 10:2025 A04 — Криптографические уязвимости (рассматривается как утечка чувствительных данных и отсутствие защиты).

Часть 1 – Утечка конфиденциальных документов через публичный файловый раздел

Шаги

1. В пользовательском интерфейсе откройте раздел «About Us» и найдите ссылку «Terms of Use» (она ведёт в каталог `/ftp`).
2. Откройте листинг FTP-каталога
3. Ищите конфиденциальные/внутренние документы. Попробуйте открыть такой файл.
4. Зафиксируйте доказательство и убедитесь, что соответствующее задание отмечено как выполненное (если оно присутствует в вашей таблице результатов).

Вопросы для обсуждения

- Почему это нельзя назвать просто «неправильной конфигурацией»? Какой уровень классификации данных был нарушен?
- Когда следует использовать шифрование, а когда достаточно контроля доступа?

Список проверок для предотвращения

- Не раскрывать внутренние файловые области неаутентифицированным пользователям.
- Классифицировать данные и применять явные политики доступа.
- Отключить листинг каталогов / авто-индексацию.

Часть 2 – Неподписанный JWT-токен

Шаги

1. Войдите в систему под любой учётной записью.
2. Найдите JWT-токен, используемый приложением для идентификации пользователя
3. Декодируйте заголовок и полезную нагрузку токена (base64url).

4. Проверьте проверяет ли сервер алгоритм подписи и саму подпись токена. Если проблема обнаружена, попробуйте модифицировать одно из полей для получения привилегированного доступа.
5. Повторно закодируйте заголовок и полезную нагрузку в base64url.
6. Используйте полученный токен в запросе и зафиксируйте результат (привилегированное поведение или решение задания).

Список проверок для предотвращения

- Отклонять токены с `alg=none`; требовать подписи и фиксировать разрешённые алгоритмы.
- Проверять `issuer`, `audience`, срок действия и подпись с использованием корректных ключей.

ЛР_5 — Пакет инъекций (SQL-инъекция + DOM-XSS)

Лабораторная работа охватывает пятый риск OWASP Top 10:2025 A05 — Инъекции.

Часть 1 — SQL-инъекция при входе (обход аутентификации)

Шаги

1. Перейдите на страницу «Login» и выполните один обычный неудачный вход, чтобы увидеть базовое поведение.
2. В поле «Email» введите строку, характерную для SQL-инъекции.
3. Посмотрите, происходит ли обход аутентификации (часто пользователь попадает в систему под первым пользователем БД).
4. Зафиксируйте доказательство (отображённый в интерфейсе идентификатор аккаунта, отметка в табло результатов и т.п.).

Часть 2 — DOM-XSS через поиск

Шаги

1. В поле «Search» проверьте есть ли возможность внедрения клиентских js скриптов и подберите рабочий payload
2. Наблюдайте выполнение скрипта (всплывающее окно «alert»). Сделайте скриншот.
3. Опишите 2–3 реальных последствия XSS (не просто «alert», а, например, кража сессионных куки-файлов, подмена содержимого страницы, фишинг).

Список проверок для предотвращения

- **SQL:** использование параметризованных запросов (избегать конкатенации строк), безопасные шаблоны ORM, минимальные привилегии для учётных записей БД.
- **XSS:** валидация входных данных, кодирование вывода, избегание опасных точек входа в DOM, применение CSP там, где это возможно.

ЛР_6 — Время расплаты (отрицательное количество → злоупотребление бизнес-логикой)

Лабораторная работа охватывает шестой риск OWASP Top 10:2025 A06 — Небезопасный дизайн.

Цель

Показать, как отсутствие серверных валидаций позволяет пользователю получить выгоду, задав отрицательное значение общей суммы корзины.

Шаги

1. Войдите в систему под любой обычной учётной записью.
2. Добавьте в корзину дорогой товар (например, самый дорогой, который найдете).
3. С помощью инструментов «DevTools» найдите запрос, который обновляет количество товара в корзине.
4. Скопируйте запрос как ``curl`/`fetch``, вставьте его в терминал (Ubuntu) и модифицируйте таким образом, чтобы нарушить логику работы приложения, например ввести отрицательное значение одного из полей вместо положительного.
5. Убедитесь, что модифицированный запрос был успешно обработан.
6. Перейдите к оформлению заказа и оплатите покупку своим кошельком. Если итог < 0 , система может зачислить деньги на ваш кошелек вместо списания.
7. Зафиксируйте доказательства: изменённый запрос, отрицательная сумма, обновлённый баланс кошелька / отметка о выполнении задания.

Вопросы для обсуждения

- В чем проблема бизнес-логики?
- Почему ограничения в клиентском интерфейсе не имеют значения?

Список проверок для предотвращения

- Выполнять серверную валидацию входных данных при обновлении корзины (минимальное/максимальное количество, только целые числа).
- При возможности добавлять ограничения в СУБД (например, ``CHECK quantity >= 1``).
- Проверять переменные при оформлении заказа: итог ≥ 0 ; списание с кошелька только положительными суммами.
- Использовать надёжные типы для денежных значений (decimal), избегать ошибок округления с плавающей точкой и централизовать логику расчётов цен.

ЛР_7 — Слабые/дефолтные учётные данные (без SQL-инъекции)

Лабораторная работа охватывает седьмой риск OWASP Top 10:2025 A07 — Сбой аутентификации.

Шаги

1. Перейдите на страницу «Login».
2. Введите адрес администратора: `admin@juice-sh.op`.
3. Попробуйте слабый/дефолтный пароль, часто используемый в учебных приложениях. (bruteforce)
4. Зафиксируйте доказательство успешного входа и выполнения задания (если оно присутствует).

Список проверок для предотвращения

- В продакшн-среде (финальной версии приложения/сервиса) не использовать дефолтные учётные данные; требовать смену пароля при первом входе.
- Политика надёжных паролей + блокировка известных утекших паролей.
- Многофакторная аутентификация там, где это уместно; ограничение частоты запросов и обнаружение аномалий.
- Защищённый процесс восстановления доступа (например, избегать угадываемых вопросов).

ЛР_8 — Небезопасная десериализация (бомба памяти)

Лабораторная работа охватывает восьмой риск OWASP Top 10:2025 A08 — Сбои в обеспечении целостности ПО и данных.

Шаги

1. Войдите в систему под любой учётной записью.
2. Перейдите в «Contact Us → Complain?» – вы окажетесь на странице `#/complain`.
3. Сначала проверьте, что устаревший путь загрузки B2B-файлов доступен через этот экран: нажмите «Choose File» и попытайтесь выбрать XML-файл размером < 100 KB.
4. Отправьте любое сообщение вместе с этим файлом.

Вы должны увидеть поведение `410 Gone` и/или сообщение: “B2B customer complaints via file upload have been deprecated for security reasons! This is the official proof that you are using the deprecated interface.”

5. Создайте файл `bomb.yml` со следующим содержимым:

```
``yaml
a: &a [_____]
b: &b [*a,*a,*a,*a,*a,*a,*a,*a]
c: &c [*b,*b,*b,*b,*b,*b,*b,*b]
d: &d [*c,*c,*c,*c,*c,*c,*c,*c]
e: &e [*d,*d,*d,*d,*d,*d,*d,*d]
f: &f [*e,*e,*e,*e,*e,*e,*e,*e]
g: &g [*f,*f,*f,*f,*f,*f,*f,*f]
h: &h [*g,*g,*g,*g,*g,*g,*g,*g]
i: &i [*h,*h,*h,*h,*h,*h,*h,*h]
...
```

6. Оставаясь на той же странице `#/complain`, введите любое сообщение жалобы.
7. Загрузите `bomb.yml` через кнопку «Choose File» (раздел «Invoice»).
8. Отправьте форму.

Почему это работает

- Уязвимый путь – устаревший B2B-запрос файловой жалобы, доступный через `#/complain`.
- YAML-бомба использует рекурсивное расширение алиасов, создавая чрезвычайно тяжёлый процесс парсинга.

- Руководство Juice Shop указывает, что «Memory Bomb» считается решённым, когда сервер занят достаточно долго; официальное решение говорит, что загрузка этой структуры заставляет обработку работать около 2 секунд до тайм-аута.

Список проверок для предотвращения

- Не десериализовать форматы, контролируемые атакующим, если это можно избежать.
- Предпочитать строгую схему с валидацией данных вместо сложного восстановления объектов.
- Ограничивать размер файлов, глубину вложенности, глубину рекурсии, время CPU и объём памяти.
- Изолировать рискованные парсеры и компоненты обработки файлов.
- Мониторить всплески задержек парсера и повторяющиеся некорректные загрузки.

ЛР_9 — Открытые метрики (утечка эндпоинта (конечной точки) наблюдаемости)

Лабораторная работа охватывает девятый риск OWASP Top 10:2025 A09 — Сбои логирования и оповещения.

Шаги

1. Найдите типичные настройки мониторинга (Prometheus обычно использует путь /metrics).
2. Проверьте, не раскрывает ли путь внутренние детали (версии, счётчики, маршруты, системную информацию).
3. Зафиксируйте доказательство и объясните, почему такие данные важны для злоумышленников.

Список проверок для предотвращения

- Защищать эндпоинты наблюдаемости аутентификацией/авторизацией и сетевым сегментированием.
- Очищать чувствительные метки и поля из метрик.
- Настроить оповещения о необычных паттернах доступа к телеметрическим эндпоинтам.

ЛР_10 (опционально) — Микролаборатория SSRF (два контейнера — docker-compose)

Лабораторная работа охватывает **риск OWASP Top 10:2021 A10** — Подделка серверных запросов (SSRF).

Важное замечание

В базовом Docker-образе Juice Shop задание SSRF недоступно. Эта опциональная микролаборатория предоставляет безопасную демонстрацию SSRF.

Запуск среды для выполнения лабораторной работы

Создайте папку с файлом docker-compose.yml. Ниже приведён безопасный `docker-compose.yml` для обучения:

```
``yaml
services:
  app:
    image: python:3.12-slim
    command:
      - sh
      - -lc
      - |
        pip install -q flask requests
        cat >/tmp/app.py <<'PY'
        from flask import Flask, request, Response
        import requests
        app = Flask(__name__)
        @app.get("/")
        def index():
            return "Try /fetch?url=http://example.com\n", 200
        @app.get("/fetch")
        def fetch():
            url = request.args.get("url", "")
            if not url:
                return "missing url\n", 400
            r = requests.get(url, timeout=3)
            return Response(
```

Commented [API1]: Это риск из owasp 2021 – в 2025 его нет (а он у нас в презе и здесь). ИИ говорит, что он теперь включён в A01 (в 2025 году), а десятый теперь **Неправильная обработка исключительных условий**

Commented [DP2R1]: Ок, поправил, оставим так

Commented [DP3R1]:

```
        r.text,
        status=r.status_code,
        content_type="text/plain",
    )
app.run(host="0.0.0.0", port=8080)
PY
python /tmp/app.py
ports:
- "8080:8080"
networks:
- labnet

internal:
image: python:3.12-slim
command:
- sh
- -lc
- |
cat >/tmp/internal.py <<'PY'
from http.server import BaseHTTPRequestHandler, HTTPServer
class H(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/meta":
            self.send_response(200)
            self.end_headers()
            self.wfile.write(b"INTERNAL_METADATA=top-secret\n")
        else:
            self.send_response(404)
            self.end_headers()
    def log_message(self, format, *args):
        pass
HTTPServer(("0.0.0.0", 9000), H).serve_forever()
PY
python /tmp/internal.py
```

networks:

- labnet

networks:

labnet: {}

'''

Запустите его через консоль:

'''bash

docker compose up

'''

Шаги

1. Откройте <http://localhost:8080/> — убедитесь, что сервис работает.
2. Выполните запрос к внешнему ресурсу:
`http://localhost:8080/fetch?url=https://example.com`
3. Выполните SSRF-запрос к внутреннему сервису:
`http://localhost:8080/fetch?url=http://internal:9000/meta`
4. Зафиксируйте полученные только для внутреннего доступа данные как доказательство.

Список проверок для предотвращения

- Использовать whitelist-списки доменов/схем; блокировать внутренние/приватные IP-диапазоны.
- Отключать перенаправления; блокировать небезопасные схемы (`file://`, `gopher://` и др.).
- Применять egress-контроль (прокси/фаервол) и защищать метаданные-эндпоинты.

ЛР_11 — Инъекция команд через переполнение буфера (коррупция данных → выполнение команд)

Лабораторная работа охватывает влияние коррекции памяти и отвечает на вопрос, почему сочетание ``system()`` и влияния атакующего опасно. Касается темы «Безопасность памяти и риски нативного кода».

Требования: Linux-VM (или WSL) с установленными ``gcc`/`clang`` и ``make``. Инструктор предоставляет исходный код ниже.

Примечание к области: В этой лаборатории намеренно не используется эксплуатация управления потоком (ROP/ret2libc). Цель — показать, как переполнение может привести к выполнению команд, повреждая соседние данные.

Цели

- Понять разницу между классической инъекцией команд и выполнением команд, вызванным переполнением памяти.
- Воссоздать нежелательное выполнение команды, переполнив буфер и затронув соседнюю строку команды.
- Исправить первопричину (границы) и убрать влияние атакующего на выполнение команды.
- Составить короткий список профилактических мер и план верификации для реальных проектов.

Модель угроз

- **Ввод атакующего:** длинная строка, подаваемая туда, где программа ожидает имя.
- **Нарушенное предположение:** строка команды считается «внутренней», но размещена рядом с пользовательским буфером.
- **Последствия:** команда, исполняемая оболочкой ОС, может быть заменена на произвольную, выбранную атакующим.

Часть 1 — Сборка и базовый запуск

| Команда | Описание |
|-------------------------------|-----------------------------|
| <code>`make cmd_vuln`</code> | собрать уязвимую версию |
| <code>`make cmd_fixed`</code> | собрать исправленную версию |

Запустите уязвимую программу интерактивно и посмотрите базовое поведение:

```
```bash
```

```
./overflow_cmd
```

```
...
```

**Ожидаемый результат:** программа выводит приветствие и запускает безопасную команду, например ``echo SAFE_COMMAND``.

## Часть 2 — Триггер выполнения команды через переполнение (только для обучения)

Мы используем детерминированный payload, который переполняет ``name[64]`` и записывает новую команду (например ``id``) в соседний буфер ``cmd[64]``.

```
```bash
```

```
python3 -c "print('A'*64 + 'id')" | ./overflow_cmd
```

```
...
```

- Наблюдайте вывод команды ``id`` (или любой другой выбранной вами команды).
- Зафиксируйте доказательство (скриншот вывода терминала).

Часть 3 — Проверка исправления

```
```bash
```

```
python3 -c "print('A'*64 + 'id')" | ./overflow_cmd_fixed
```

```
...
```

**Ожидаемый результат:** команда ``id`` не будет выполнена; всегда будет запущена только фиксированная безопасная команда. Зафиксируйте вывод.

## Вопросы для обсуждения

- Почему это всё равно «инъекция команд», хотя ввод не конкатенируется напрямую со строкой команды?
- Что является основной уязвимостью: использование ``system()`` или переполнение буфера? Почему?
- Каков был бы реальный риск, если такой код был доступен по сети (например, в виде демона/службы)?

## Список проверок для предотвращения

- По возможности использовать языки с безопасной обработкой памяти (memory safe languages).
- Всегда задавать корректные границы для функций ввода (например, ``fgets`` с ``sizeof``, проверка длины).
- Не размещать чувствительные данные рядом с пользовательскими буферами в памяти (уменьшить риск «смежных переменных»).



- Избегать `system()` и интерпретации оболочки; предпочтительно использовать семейство `exec` с фиксированным массивом аргументов (или полностью убрать выполнение команд ОС).
- Компилировать с включёнными механизмами защиты (stack protector, PIE, RELRO, FORTIFY) и запускать сборки с санитайзерами в CI.
- Добавлять тесты границ и фаззинг для нативных парсеров и обработчиков протоколов.

### Исходный код

``overflow_cmd.c`` — **намеренно уязвимый** (переполнение затрагивает соседний ``cmd``)

```
```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/*
  _INTENTIONALLY VULNERABLE (TRAINING ONLY)_
  _DATA-ORIENTED exploit demo:_
  - пользовательский ввод попадает в name[64]
  - переполнение затрагивает соседний cmd[64]
  - cmd затем передаётся в system(), позволяя выполнить произвольную команду
  NOTE: демонстрируется без эксплуатации управления потоком (ROP/ret2libc).
*/

typedef struct {
    char name[64];
    char cmd[64];
} Frame;

int main(void) {
    Frame f;
    memset(&f, 0, sizeof(f));

    /* базовая безопасная команда */
    strcpy(f.cmd, "echo SAFE_COMMAND");
```

```

puts("overflow_cmd: enter your name:");
puts("(Hint: long input can corrupt adjacent variables)");

/* VULNERABLE: читает до 1023 байт в буфер из 64 байт → переполняет f.cmd */
if (fgets(f.name, 1024, stdin) == NULL) {
    puts("Input error.");
    return 1;
}

/* удаляем перевод строки */
f.name[strcspn(f.name, "\n")] = '\0';

/* если переполнение записало перевод строки в cmd, отрезаем его */
f.cmd[strcspn(f.cmd, "\n")] = '\0';

printf("Hello, %s\n", f.name);
printf("About to run: [%s]\n", f.cmd);

/* опасно в реальных приложениях: выполнение оболочки с данными, контролируемые атакующим */
(void)system(f.cmd);

return 0;
}
...

```

``overflow_cmd_fixed.c`` — **исправленная версия** (корректные границы, отсутствие влияния атакующего)

```

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/*

```

```

FIXED VERSION

Исправлено переполнение и убрано влияние атакующего на выполняемую команду:

- ввод читается с правильными границами (sizeof)
- команда фиксирована (в реальных приложениях лучше вовсе избавиться от system())
*/

int main(void) {
 char name[64];

 puts("overflow_cmd_fixed: enter your name:");

 if (fgets(name, sizeof(name), stdin) == NULL) {
 puts("Input error.");
 return 1;
 }
 name[strcspn(name, "\n")] = '\0';

 printf("Hello, %s\n", name);

 /* команда фиксирована для обучения; в продакшн-коде следует избегать system() */
 (void)system("echo SAFE_COMMAND");

 return 0;
}
...

```

### Цели `Makefile` (добавьте в ваш Makefile)

```

``make

--- L12: command execution via data corruption demo ---

cmd_vuln: overflow_cmd.c
 $(CC) $(CFLAGS_BASE) $(CFLAGS_VULN) $< -o overflow_cmd $(LDFLAGS_BASE) $(LDFLAGS_VULN)

cmd_fixed: overflow_cmd_fixed.c

```

`$(CC) $(CFLAGS_BASE) $< -o overflow_cmd_fixed $(LDFLAGS_BASE)`

...